Practical No-05: Image Generation using Pretrained Stable Diffusion

Student: Sujal Chaudhari
PRN: 202401110026

```
%pip install -q -U diffusers transformers accelerate safetensors matplotlib
```

```
import time
import torch
import matplotlib.pyplot as plt

from diffusers import (
    StableDiffusionPipeline,
    PNDMScheduler,
    DDIMScheduler,
    EulerDiscreteScheduler,
    EulerAncestralDiscreteScheduler,
    DPMSolverMultistepScheduler
)

print("All libraries imported successfully!")
print("PyTorch version:", torch.__version__)
```

```
All libraries imported successfully!
PyTorch version: 2.11.0+cu130
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

if device == "cpu":
    dtype = torch.float32
else:
    dtype = torch.float16

pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=dtype,
    use_safetensors=True
)

pipe = pipe.to(device)

# Save scheduler configuration for the experiments
scheduler_config = pipe.scheduler.config

print("Model loaded successfully.")
print("Scheduler:", type(pipe.scheduler).__name__)
```

```
Using device: cuda
/usr/local/lib/python3.13/dist-packages/diffusers/utils/deprecation_utils.py:23: FutureWarning: `torch_dtype` is deprecated and
  deprecate("torch_dtype", "1.0.0", _TORCH_DTYPE_DEPRECATION_MESSAGE)
model_index.json: 100%  541/541 [00:00<00:00, 36.3kB/s]
Download complete:  4.75GB,  330MB/s
Reconstruction complete: 100%  5.48GB / 5.48GB,  215MB/s
Fetching 15 files: 100%  15/15 [00:27<00:00,  1.44s/it]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Loading pipeline components...: 100%  7/7 [00:23<00:00,  4.53s/it]
Loading weights: 100%  196/196 [00:01<00:00, 125.42it/s]
Loading weights: 100%  396/396 [00:04<00:00, 82.97it/s]
Model loaded successfully.
Scheduler: PNDMScheduler
```

```
def generate(prompt, steps=30, guidance=7.5, seed=42):
```

```
    generator = torch.Generator(
        device=device
    ).manual_seed(seed)

    # Synchronize GPU before timing
    if device == "cuda":
        torch.cuda.synchronize()

    start = time.perf_counter()

    with torch.inference_mode():
        result = pipe(
            prompt,
            num_inference_steps=steps,
            guidance_scale=guidance,
            generator=generator
        )

    if device == "cuda":
        torch.cuda.synchronize()

    seconds = time.perf_counter() - start

    return result.images[0], seconds
```

```
import matplotlib.pyplot as plt
import math

def show_images(images, titles, columns=3):
    if not images:
        print("No images available to display.")
        return

    rows = math.ceil(len(images) / columns)

    fig, axes = plt.subplots(
        rows,
        columns,
        figsize=(5 * columns, 5 * rows),
        squeeze=False
    )

    for i, (image, title) in enumerate(zip(images, titles)):
        ax = axes.flat[i]
        ax.imshow(image)
        ax.set_title(title)
        ax.axis("off")

    # Hide unused subplot positions
    for i in range(len(images), rows * columns):
        axes.flat[i].axis("off")

    plt.tight_layout()
    plt.show()

print("show_images() function defined successfully!")
```

```
show_images() function defined successfully!
```

```
prompt = (
     "A beautiful underwater coral reef filled with colorful fish, "
    "sunbeams penetrating clear blue water, sea turtles, "
    "vibrant marine life, realistic underwater photography"
)

schedulers = {
    "PNDM": PNDMScheduler,
    "DDIM": DDIMScheduler,
    "Euler": EulerDiscreteScheduler,
    "Euler Ancestral": EulerAncestralDiscreteScheduler,
    "DPM-Solver++": DPMSolverMultistepScheduler,
}

images = []
titles = []
scheduler_results = []
```

```
for name, scheduler_class in schedulers.items():

    pipe.scheduler = scheduler_class.from_config(
        scheduler_config
    )

    img, sec = generate(
        prompt,
        steps=30,
        guidance=7.5,
        seed=42
    )

    images.append(img)
    titles.append(f"{name} ({sec:.1f} s)")

    scheduler_results.append({
        "Scheduler": name,
        "Inference Steps": 30,
        "Guidance Scale": 7.5,
        "Seed": 42,
        "Generation Time (seconds)": round(sec, 2)
    })

show_images(images, titles)

import pandas as pd

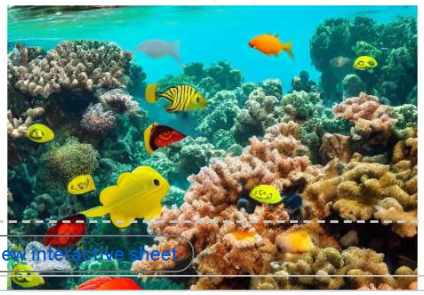
scheduler_df = pd.DataFrame(scheduler_results)
display(scheduler_df)
```

```
100%  30/30 [00:04<00:00,  6.34it/s]
100%  30/30 [00:04<00:00,  6.18it/s]
100%  30/30 [00:04<00:00,  6.20it/s]
100%  30/30 [00:04<00:00,  6.03it/s]
100%  30/30 [00:05<00:00,  5.73it/s]
```

PNDM (5.2 s)



DDIM (5.1 s)



Euler (5.1 s)



Next steps: Generate code with scheduler_df New interactive sheet

Start coding or generate with AI.







Euler Ancestral (5.3 s)



DPM-Solver++ (5.5 s)



Scheduler	Inference Steps	Guidance Scale	Seed	Generation Time (seconds)

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.